

Athantor: Code-Verified Hypothesis Refinement with an Independent Artifact Reviewer for ARC-AGI-2

Dastin Huang*
Independent Researcher

June 6, 2026

Abstract

We present **Athantor**, a stateful multi-agent system for the ARC-AGI-2 abstract reasoning benchmark. On the public evaluation set, Athantor scores **95.7%** at an average inference cost of **\$3.12 per task** — the lowest cost-per-task among systems above 95% accuracy on the public evaluation, and approximately 2× cheaper than the strongest comparable-accuracy system that achieves its score without executable verification (Squeeze-Evolve, 97.5% at \$5.93/task). Three interlocking mechanisms drive this efficiency. First, exploratory Python execution is repurposed as a token-efficient *verification* tool, separated from final solution submission; short executable snippets replace lengthy chain-of-thought enumeration. Second, an *artifact-only reviewer* based on a fresh-context Gemini 3.1 Pro examines the solver’s hypothesis text, candidate code, predicted outputs, and training accuracy — never the solver’s reasoning chain — and returns one of APPROVE, REJECT, or EXPAND_CANDIDATES. Third, *Inter-Context Artifact Exchange* (ICAE) compresses solver state into a portable checkpoint at context boundaries or on reviewer rejection, allowing multi-iteration discovery within bounded context windows. We release 119 full inference traces alongside the implementation as an auditable record.

1 Introduction

The ARC-AGI-2 benchmark [6], a sequel to the original Abstraction and Reasoning Corpus [5], measures a system’s ability to infer transformation rules from a small number of input/output examples and apply them to held-out test grids. The benchmark is designed to resist memorization and reward genuine abstraction, and it has remained substantially harder than language-only or vision-only benchmarks for current frontier models, including under chain-of-thought prompting [25].

Recent progress on ARC-AGI-2 has come from several distinct families of agentic systems, which Section 2 surveys in detail. Public-eval scores above 95% have been reported for systems built on program synthesis with voting [7], evolutionary recombination over candidate programs [22], and evolutionary search with population-level review [13], all of which operate at \$5–\$12 per task. Multi-model iterative refinement [17] has prioritized verified semi-private submission and reportedly holds the highest verified semi-private score at 54%. Each of these systems organizes accuracy gains around *breadth of search*: more candidates, more generations, more parallel attempts. Their cost scales with the population dimension, and marginal accuracy improvements demand proportionally larger inference budgets.

The question this paper addresses is whether a fundamentally different organization of the same scaffolding budget — one that invests in *single-thread depth* rather than population breadth — can match 95%+ accuracy at substantially lower cost. The answer we report is

*dastin0359@gmail.com. Personal research project; not affiliated with any institution’s ARC-AGI work.

affirmative: **95.7%** on the ARC-AGI-2 public evaluation at **\$3.12 per task**, the lowest cost-per-task among systems above 95% and approximately 2–4× cheaper than every other published system at comparable accuracy.

This paper presents the system behind that result. Instead of broadening the search, Athanor narrows it: a single solver thread maintains a research-style internal state, executes short verification snippets to falsify candidate sub-rules, and submits a hypothesis-plus-code artifact to an independent reviewer that has never seen its reasoning chain. Threshold-triggered context compression preserves the solver’s research state across context-window boundaries, allowing iteration depth that a single fixed context could not support.

Contributions.

- A solver design that uses Python execution as *verification* of sub-rules rather than as a final-program scorer. We document the token-efficiency consequences of this choice.
- An independent *artifact-only reviewer*: among the systems we are aware of that score above 95%, this is the only mechanism that can reject a solution that passes the training examples on generalization grounds.
- *Inter-Context Artifact Exchange* (ICAE): an atomic compression-and-handover step triggered both by context pressure and by reviewer rejection, allowing multi-iteration discovery within a single solver thread.
- A public release of 119 full inference traces (one per puzzle, one excluded for data validity) as an auditable record of per-task scoring.

Scope. This paper reports public-evaluation results only. We have not submitted to the ARC-AGI-2 semi-private set; the headline score therefore reflects the public configuration on the public puzzles. Section 4.3 describes the per-run compliance and cross-run selection methodology, both of which are relevant to interpreting the score.

2 Related work

We survey four families of agentic systems that have reached the upper accuracy band on ARC-AGI-2, organized by mechanism rather than by score, then place them against the ceiling of pure chain-of-thought (CoT) prompting. All numbers below are public evaluation unless otherwise noted; verified semi-private numbers are flagged because the two regimes are not directly comparable across systems and submission methodologies. A summary of the agentic systems appears in Table 2 (Section 4.1); the discussion here is qualitative and aims to surface the design choice each system makes, the role each assigns to code execution, and the kind of quality gate each applies before accepting a candidate solution.

The CoT-only ceiling. The ARC Prize Foundation maintains a public corpus of attempts on the public evaluation set, comprising 10,529 logged runs from 58 well-attempted single-model configurations as of mid-April 2026 [3]. These configurations are CoT-only: the model is prompted with the puzzle and emits a test prediction directly, with no scaffolding, code execution, voting, or external review. Across the full corpus, no CoT-only configuration exceeds 88.1% public-eval accuracy at any cost (the top entry being Gemini 3.1 Pro), and no `claude-opus-4-6` CoT configuration exceeds 79.0%. We refer to this corpus in Section 4.2 as the empirical ceiling against which to measure the contribution of an agentic scaffold over the same base model.

2.1 Program synthesis with voting: Confluence

Confluence [7] synthesizes a population of candidate Python programs from the puzzle’s training pairs and aggregates their predictions on the held-out test grids by voting. Each candidate is generated by a separate prompted LLM call and scored by execution against the training grids; the test prediction is the modal output of the surviving population. Confluence reports 97.9% on ARC-AGI-2 public evaluation at \$11.77 per task. The mechanism is strong on accuracy because voting averages out individual generation noise, but the cost scales linearly with population size and the quality gate is *cross-candidate*: a single candidate that passes all training pairs but fails to generalize cannot be rejected on its own merits, only outvoted.

2.2 Evolutionary recombination without execution: Squeeze-Evolve

Squeeze-Evolve [22] evolves candidate solutions through crossover and mutation in pure LLM token space, with no code execution at any stage. Candidates are scored by training-grid match, also computed by the LLM itself, and the population is filtered by intrinsic confidence routing across model tiers. The system reaches 97.5% public-eval accuracy at \$5.93 per task, demonstrating that high ARC-AGI-2 scores are achievable without executable verification. We treat Squeeze-Evolve as the relevant cost baseline against which to measure the value of code execution: any system that uses code and reports a lower cost than \$5.93 is recovering tokens that Squeeze-Evolve spends on per-step verification in pure reasoning.

2.3 Evolutionary search with population review: Imbue Darwinian Evolver

The Imbue Darwinian Evolver [13] maintains a population of candidate programs and selects survivors based on training accuracy plus LLM-judged generalization. Code is executed, but in the role of a *final-program scorer* — a generated `solve()` runs against the training pairs and the resulting accuracy contributes to fitness. The published configuration that uses Gemini 3.1 Pro reports 95.1% at \$8.71 per task; a lower-cost configuration that uses Gemini Flash reaches 61.4% at \$2.42, illustrating that the population-search approach degrades gracefully across model tiers. The quality gate is again population-relative: candidates compete against one another by fitness, with no mechanism that can reject a single fit-saturating candidate on independent generalization grounds.

2.4 Multi-model iterative refinement: Poetiq

Poetiq [17, 18] takes a different organizational choice: rather than committing to one base model, the system is presented as a “meta” refinement loop that selects among multiple LLMs (GPT, Claude, Gemini, Grok, GPT-OSS) to navigate a Pareto frontier of cost and accuracy. Within each puzzle, the loop generates a candidate (sometimes code, sometimes natural-language output), inspects feedback, and iteratively refines until a self-audit terminates the process. Poetiq’s public material reports their submission as “top of the official leaderboard” with “half the cost” of the prior state of the art and with “fewer than two requests on average” per task; specific public-eval percentages are visualized rather than tabulated. Poetiq is distinctive among the systems we survey for prioritizing *verified* semi-private submission (54%, the highest verified semi-private score we are aware of) over public-eval benchmarking, and for treating model choice as a search axis rather than a fixed configuration. The system is open-source [18].

2.5 This work

Athamor differs from each of the above on three axes. First, the loop is organized around *single-hypothesis refinement* rather than population search: one solver thread maintains research-style internal state and refines a single hypothesis across iterations, instead of generating, scoring,

and recombining many candidates in parallel. Second, code execution plays a verification role rather than a scoring or synthesis role — the price of doing this is the design effort of separating exploratory and final execution; the empirical return is a $\sim 2\times$ cost reduction against Squeeze-Evolve at comparable accuracy (Section 4). Third, the quality gate is an independent artifact-only reviewer (Section 3.2); among systems above 95% public-eval accuracy, this is the only mechanism we are aware of that can reject a single candidate that passes all training pairs on independent generalization grounds.

3 Method

Athamor is organized as a solver agent, an independent reviewer agent, and an orchestrator that mediates artifact exchange between them. The solver explores a puzzle through hypothesis text, exploratory code, and candidate code; the reviewer reads only the resulting artifacts and returns a verdict; the orchestrator triggers context compression on a size threshold or on reviewer rejection.

Throughout this paper we use *artifact* for any object that crosses a context boundary — hypothesis text, `solve()` code, candidate test predictions, training accuracy, reviewer verdict, or distilled research-state checkpoint — as distinct from a *reasoning chain*, which never leaves the context that produced it.

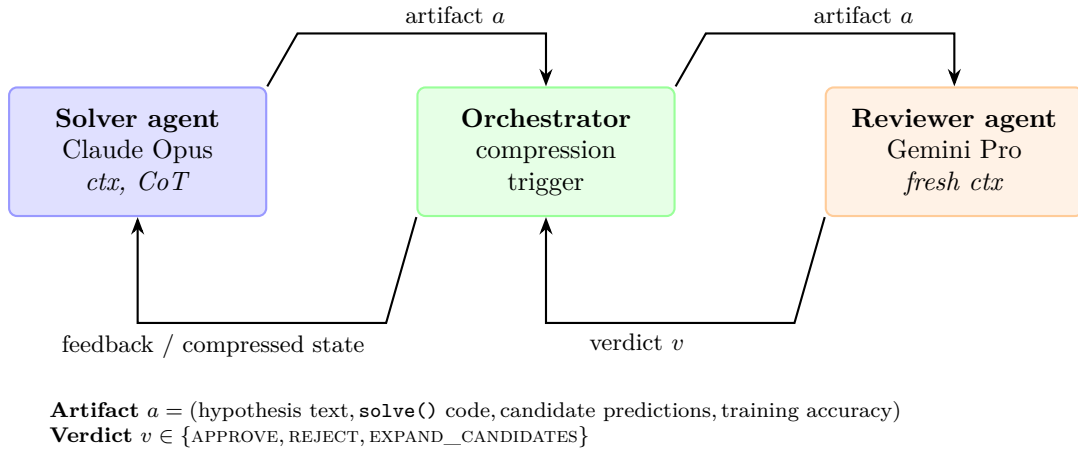


Figure 1: System pipeline. The solver (Claude Opus) and reviewer (Gemini 3.1 Pro, fresh context per puzzle) exchange only artifacts; reasoning chains stay local to each agent. The orchestrator routes artifacts and triggers compression (on a context-size threshold or on REJECT).

The solver maintains three explicitly typed objects, summarized in Table 1. This typing matters because each object plays a different role downstream: the hypothesis externalizes the solver’s current rule for review, the exploration code falsifies sub-rules without commitment, and the transform code is the only object that is graded against training pairs and used to generate test predictions.

Object	Submitted via	Counts toward iteration budget?
Exploration code	<code>run_code</code>	No
Transform hypothesis	<code>submit_transform_hypothesis</code>	No
Transform code	<code>execute_python_solution</code>	Yes

Table 1: The three solver-side objects. Exploration code is unlimited and used for hypothesis testing; transform code is the graded artifact.

3.1 Code execution as verification, not as program scoring

The first design choice is to treat exploratory Python execution as a tool for *checking* a hypothesis rather than for *producing* the final program. Concretely, exploration code (`run_code`, `run_code_in_previous_runtime`) is unlimited and never enters the iteration budget; transform code (`execute_python_solution`) is graded against training pairs and counts as one iteration per call.

The motivation is token efficiency. Many sub-rule checks that arise during ARC reasoning have the form “does property ϕ hold across all training inputs?” — for example, *is every row monochromatic* or *does the output palette equal the input palette*. In a pure-reasoning system, verifying ϕ requires the model to enumerate cells of each input grid in token space, prone to arithmetic slips and consuming hundreds of tokens per check. In Athanor, the solver writes the check directly. Listing 1 shows a real example, taken verbatim from the trace of a solved puzzle: the solver needs to know which rotational symmetries leave a particular colored sub-pattern fixed.

Listing 1: Exploratory verification from release run 6ffbe589, iteration 0 (full version with comments in Appendix A). The solver checks which 90° rotations leave a colored sub-pattern fixed; the four-row truth table is consumed directly, without further reasoning tokens spent on per-cell rotation arithmetic.

```
mask_in = (np.array(train_samples[0]['input']))[0:13, 6:19] == 6)
for k in range(4):
    print(f"k={k}:", np.array_equal(np.rot90(mask_in, k), mask_in))

# stdout: k=0 True ; k=1 False ; k=2 True ; k=3 False
```

The output is a four-line truth table over a deterministic predicate. Whatever follows in the solver’s reasoning can treat each rotation’s invariance as established or falsified, with no further verification tokens spent.

Token cost. The exploration call above sends ~ 57 input tokens of code and receives ~ 39 output tokens of result, for a round-trip of roughly 100 tokens. A pure-reasoning equivalent would have to mentally rotate a 13×13 mask four times and compare cell-by-cell — realistically several hundred tokens per rotation, with non-trivial arithmetic-error risk on the rotation indexing alone.¹ The qualitative observation generalizes: invariant checks of the form “is property ϕ stable under transformation T ?” are dramatically cheaper to express as code than as chain-of-thought enumeration. Hard ARC puzzles typically require dozens of such checks, so the per-check savings compound.

Two execution modes. A complementary design choice is that the solver is given *two* execution tools, not one. The first, `run_code`, is one-off: each call spawns a fresh runtime, imports re-execute, and any defined symbol disappears at the end of the call. The second, `run_code_in_previous_runtime`, is persistent: it reuses a long-lived Python process across the entire puzzle, with the data structure of a Jupyter session [14] — imported modules, helper functions, parsed grids, and accumulated analysis state survive between calls. Persistent execution is offered natively by Gemini’s server-side code interpreter [9]; on the Anthropic tool-use API used here, the runtime is implemented locally inside the orchestrator’s tool handler.

The two modes serve different purposes. Persistent execution amortizes setup cost across the dozens of checks a hard ARC puzzle requires: once `numpy` is imported and the training grids are parsed and bound to local variables, every subsequent invariant check is just the predicate

¹We do not report this as a measured average across all exploration calls; it is one concrete example. A full per-call breakdown is appropriate future work.

itself, often under fifty tokens of code. One-off execution provides state hygiene — a clean slate when the solver suspects accumulated state may be coloring a result, or when testing whether a hypothesis holds independent of any helpers built up earlier in the conversation. Across all 119 release traces, 5,479 exploration calls were issued, of which 88.5% (4,847) were persistent and 11.5% (632) one-off. Persistent execution is the dominant idiom; one-off is used selectively, and the solver chooses between them on a per-call basis.

This design choice extends the cost-efficiency argument: the per-check token economy described above only holds if exploration does not pay re-import and re-parse cost on every invocation. Without a persistent runtime, the ~ 100 -token round-trip of the rotation-invariance example would be padded by hundreds of tokens of boilerplate per call — enough to erode the gap against pure-reasoning verification.

Relation to other code-using approaches. This is a design choice about the *role* of execution, not about whether to use it at all. Three other patterns appear in the literature. Code can be the chain of thought itself, as in Program-Aided Language Models [8] and Program-of-Thoughts [4], where the model emits a program whose execution *is* the answer. Code can be the final artifact under search, as in Confluence [7] and the Imbue Darwinian Evolver [13], where candidate `solve()` programs are generated and ranked by training-grid accuracy. Or code can be omitted entirely: Squeeze-Evolve [22] reaches 97.5% public-eval accuracy with no execution at all. Athanor’s contribution is a fourth point: code execution as a *verification side-channel* threaded through the reasoning loop, separate from the candidate that is ultimately submitted. Squeeze-Evolve is the relevant cost baseline — it isolates the price of verification in pure tokens — and the $\sim 2\times$ cost reduction Athanor reports against it is the empirical value of moving that verification step into Python. Figure 2 summarizes the four positions.

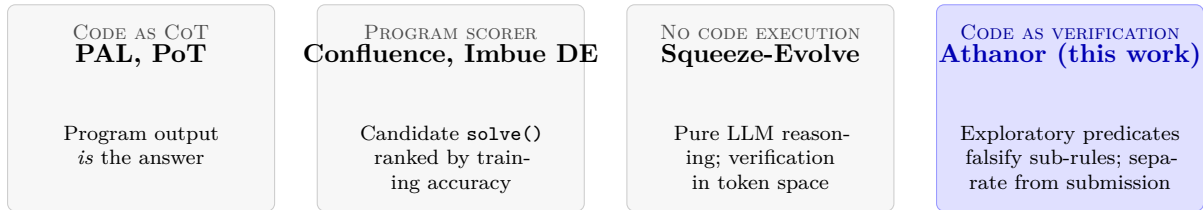


Figure 2: Four roles code execution plays in ARC-AGI-2 systems. Athanor (rightmost, highlighted) uses code as a *verification side-channel* threaded through the reasoning loop, distinct from the three other patterns in the literature.

3.2 Artifact-only independent reviewer

When the solver submits a candidate `solve()` that passes all training pairs, control transfers to an independent reviewer running in a fresh model context. We use Gemini 3.1 Pro for the reviewer; the solver runs on Claude. The two agents do not share weights, conversation, or hidden state.

The reviewer sees four objects and only those four:

1. the natural-language transform hypothesis (the solver’s externalized rule),
2. the candidate `solve()` code,
3. the candidate’s test predictions,
4. training accuracy on the puzzle’s training pairs.

The solver’s reasoning chain — the running internal monologue, the exploratory snippets and their outputs, the partial dead ends — is excluded by construction.

Design rationale. Two choices distinguish this setup from a generic “second model checks the first” pattern.

Artifact-only. If the reviewer sees the solver’s reasoning chain, an articulate-but-wrong justification can anchor the verdict toward acceptance: the reviewer ends up evaluating whether the solver’s argument is internally consistent rather than whether the solver’s rule is correct. Restricting to artifacts forces the reviewer to reconstruct the puzzle’s underlying rule independently before comparing it to the candidate. The trace excerpt below illustrates this directly.

Different model family. The solver runs on Claude Opus; the reviewer runs on Gemini 3.1 Pro. The choice of family is not incidental. Models within a family share training corpora, RLHF pipelines, and architectural lineage; smaller variants are commonly distilled from larger ones [11], with Google having documented online distillation as the training procedure for smaller Gemini variants relative to their Pro-tier counterparts [10], and Anthropic’s Claude tier structure (Haiku, Sonnet, Opus) is similarly framed as variants of a shared family [1]. Failure modes consequently correlate within a family, including subtle systematic ones that show up only on edge cases. Cross-family review trades some prompt-portability for a lower probability that solver and reviewer share the same blind spot on the same puzzle.

The reviewer returns one of three verdicts:

approve. The hypothesis explains all training data, the code implements it faithfully, and the rule appears to generalize. The candidate is accepted.

reject. The rule is wrong, overfit to the training pairs, or unsupported by the data. Rejection triggers an atomic compression-and-feedback step (Section 3.3) and the solver retries from a fresh context.

expand_candidates. The rule appears sound but legitimate ambiguity remains between two or more plausible branches. The solver is asked to emit a second candidate test output, exercising the per-test 2-attempt allowance.

The reviewer maintains a multi-turn conversation across the puzzle: the first turn receives the full puzzle data alongside the artifacts; later turns see only the *changes* since the previous review. This avoids re-anchoring on already-established context and keeps the reviewer focused on what the solver modified in response to a previous REJECT.

The transferable observation is structural. Among ARC-AGI-2 systems above 95% public-eval accuracy, every other quality gate operates over a population: Confluence aggregates votes across synthesized candidates [7], the Imbue Darwinian Evolver scores by fitness with population-level corroboration [13], and Squeeze-Evolve routes by intrinsic confidence [22]. None of these mechanisms can reject a single training-100% candidate on generalization grounds, because their gate is a comparison or aggregation across multiple candidates rather than an independent assessment of one. The artifact-only reviewer is, to our knowledge, the only mechanism among published ARC-AGI-2 systems that can.

Trace excerpt. A concrete instance from release run `faa9f03d` illustrates the mechanism. On iteration 3, the solver submits a candidate that passes all four training pairs (training accuracy $4/4 = 100\%$). The reviewer, working only from the artifact, issues REJECT — identifying two over-constrained heuristics in the candidate’s code, citing the specific buggy classification check and the unconstrained gap-fill loop, and explicitly confirming a third subroutine (intersection swapping) as correct (Listing 2; full $\sim 4,000$ -character response in Appendix B).

Listing 2: Sketch of the reviewer’s response on `faa9f03d` iteration 3.

An analysis of the training examples reveals a beautifully consistent set of rules [...] but your current implementation contains two critical over-constrained heuristics that cause it to completely fail on Test 0.

1. *The Flip Marker Logic is Sound, but the Classification is Buggy:*
[...] Marker 2 is a valid part of the line.
2. *The Gap Fill Logic is Unconstrained and Undoes Flips:* [...]
3. *The Intersection Swapping Logic is Perfect:* [...]

VERDICT: REJECT

Two properties of this response are worth highlighting. First, the reviewer arrives at the verdict by reconstructing the puzzle’s underlying rule independently and then comparing it to what the solver’s code actually does — not by post-hoc voting or by ranking against alternative candidates [26]. Second, the reviewer’s affirmation that “The Intersection Swapping Logic is Perfect” shows that the verdict is not a blanket reject of the artifact: the next iteration is given specific corrective targets while preserving the parts that already work, unlike self-refinement loops in which a single agent both critiques and revises its own output [16, 21].

3.3 Inter-Context Artifact Exchange

The third mechanism, Inter-Context Artifact Exchange (ICAE), addresses three distinct pressures that any agent doing extended single-task search encounters as its conversation lengthens. We motivate the design by working through each pressure in turn before describing the mechanism itself.

(i) Hard context-window limits. Every frontier model exposes a finite input window. The Claude Opus 4.6 configuration used here supports up to 1 million input tokens at standard pricing as of March 2026, with comparable windows on contemporaries such as Gemini 3.1 Pro. At Opus 4.6’s initial February 2026 release, however, the default window was 200,000 tokens, and the 1 million-token mode was available only behind an extra-cost beta header; general availability at standard pricing followed on March 13, 2026 [2]. Athanor’s compression threshold of $\tau = 170,000$ tokens was originally calibrated to sit comfortably inside that initial 200,000-token default window, and was retained at the same value after the 1 million-token mode became generally available — because pressures (ii) and (iii) below already made early compression desirable on its own merits. In the current configuration, the threshold therefore fires at roughly 17% of the model’s nominal window: we do not wait for the wall, and we do not move the threshold when the wall moves further away.

(ii) Compute scaling within the window. Even when the window is far from full, the marginal cost of generating each new token grows with context length, in two distinct regimes. The *prefill* step that ingests new prompt tokens or replays a fresh context is $O(N^2)$ in sequence length [23], since each token attends over all preceding tokens. *Decode* with a KV cache is cheaper per step — attention for each new token reduces to $O(N)$ because the keys and values of all preceding tokens are cached — but this still means decode throughput (tokens per second) drops with the length of the cache. The trend persists even on aggressively optimized inference stacks: SGLang and Miles Team [20] report DeepSeek-V4 decode throughput falling from 199 to 180 tok/s on B200 hardware and from 266 to 240 tok/s on H200 hardware as the context grows from 4K to 900K ($\sim 10\%$ each, with hybrid sparse attention and prefix caching enabled); production stacks without these specific optimizations exhibit substantially larger drops, and a careful throughput-vs-context analysis at scale appears in Pope et al. [19]. In an agentic loop with growing conversation, both regimes contribute: each new turn’s prefill is more expensive than the last, and every decoded token within that turn is more expensive than its predecessor. An extended single-context run therefore pays a continually increasing per-token cost long before any hard limit is reached.

(iii) **Attention quality within the window.** Liu et al. [15] document the “lost-in-the-middle” phenomenon: on tasks that require integrating evidence from a long context, model accuracy is highest when the relevant information sits at the start or end of the context and substantially lower when it sits in the middle, with the gap widening as total context length grows. The same useful evidence is *less effectively used* simply because it is buried among more surrounding tokens. Compressing the working state into a short, structurally prominent artifact at the head of a fresh context counteracts this directly: what survives compression occupies the privileged early-context positions of the new conversation.

From motivation to mechanism. These three pressures argue for an active state-management strategy: rather than letting the conversation grow until pressure (i) forces emergency truncation, the orchestrator pre-empts pressures (ii) and (iii) by triggering compression at a fixed threshold $\tau = 170,000$ tokens — well inside the underlying model’s 1 million-token window. The threshold sitting so far below the limit is itself a design choice: we do not wait for the wall. Compression is artifact-producing rather than information-discarding: the running context is distilled into a fixed-schema research-state checkpoint, and a fresh context resumes from that checkpoint plus the original puzzle data.

We refer to this exchange as Inter-Context Artifact Exchange (ICAE) and identify three modes in this system, named by the producer→consumer pair. Figure 3 traces all three modes through a representative two-cycle solve and lists the contents of each handover; the paragraphs that follow surface the points the figure does not encode.

The figure encodes the topology and contents of every handover; three points about its operation are not visually obvious and warrant text. *First*, the schema bullets in the Mode A annotations are emitted by the solver itself in response to the compression prompt, not added post-hoc by the orchestrator; this is what makes the checkpoint a research-state artifact rather than a runtime-side truncation.² The pattern is structurally similar to memory hand-offs in long-running LLM agents [21, 24], but with a fixed schema tuned for hypothesis-refinement search rather than free-form reflection. *Second*, although Mode B is drawn as a single arrow per submission, the reviewer’s persistent context anchors on these artifacts across rounds: the first submission of a puzzle arrives alongside the full puzzle data; subsequent submissions deliver only the *differences* in the artifact since the previous review, which prevents the reviewer from re-anchoring on already-established context. *Third*, the atomicity of Mode C followed by Mode A on REJECT is load-bearing: without it, the next solver step would attempt a revision in a context already near τ , carrying along the very exploratory branches the reviewer flagged as incorrect. Coupling the two operations forces the solver to distill what it learned — including the critique — before resuming.

Cross-cutting properties. The three active modes share two structural properties. *Active*: every compression is initiated by the running context when the orchestrator detects context pressure or a REJECT, not by runtime-side truncation. *Artifact-based*: every cross-context transfer is an explicit artifact (distilled state, four-tuple submission, or verdict), never a raw transcript dump. The full loop is sketched in Algorithm 1.

The structural distinction from prior work is that ICAE is neither a population memory (as in Squeeze-Evolve and the Darwinian Evolver, where state lives across many parallel candidates) nor an agent-local conversation (as in Confluence, where each synthesized program runs to completion in its own thread). It is a single-thread continuation mechanism in which the unit of state is the artifact, not the transcript.

Trace excerpt. Appendix C reproduces a full ~10,000-character distilled research state from release run faa9f03d, written by the solver immediately before the first context reset. Its schema

²Appendix C reproduces a full ~10,000-character distilled research state from a release run verbatim.

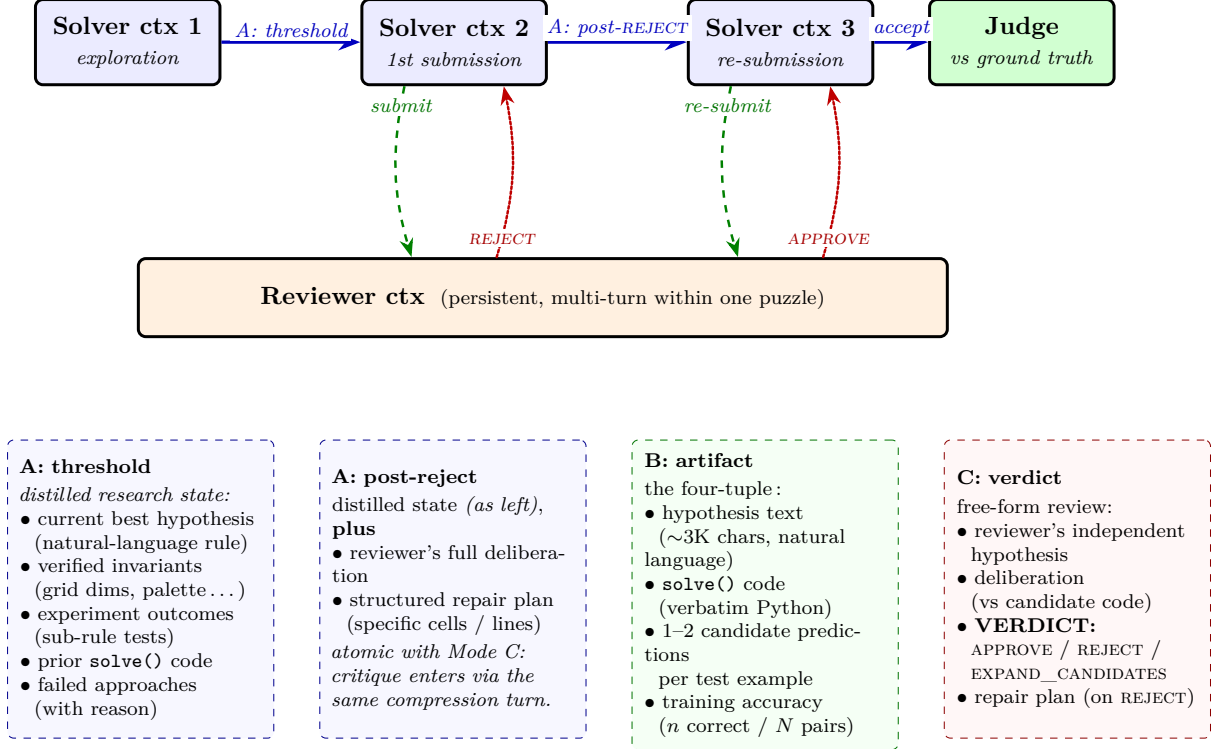


Figure 3: The three modes of Inter-Context Artifact Exchange (ICAE), traced through a representative two-cycle solve. Each top-row blue box is a *context-bounded model instance* of the solver; the solver’s context is reset at the compression threshold τ (Solver ctx 1→2, **Mode A** solid) and again on reviewer REJECT (Solver ctx 2→3, **Mode A** atomic with the rejection). The bottom orange box is the reviewer’s persistent multi-turn context, which receives solver artifacts (**Mode B** dashed) and returns verdicts to the submitting solver instance (**Mode C** dotted). The dashed annotation boxes list the verbatim contents of each handover. The rightmost **Judge** node is the external ARC-AGI-2 grader against ground-truth test outputs; it is not part of Athanor and runs only after the reviewer issues APPROVE. There is no Mode D: reviewer output is bounded and never reaches the threshold τ , so reviewer-to-reviewer self-handover is never required.

headers (*Verified Invariants, Experiments & Learnings, Hypothesis Evolution, Failed Approaches, Current State & Next Steps*) are emitted by the solver itself in response to the compression prompt; they are not post-hoc annotations. The compressed checkpoint is roughly 25× shorter than the raw conversation it replaces (~10 K characters versus the ~170 K-character threshold that triggered the compression), yet it preserves the verified invariants, the learning chronology, and the failure record. The next solver context window resumes from this checkpoint plus the original puzzle data, and the trace shows the solver immediately picking up its hypothesis refinement on iteration 1 rather than re-deriving the invariants it already established.

4 Results

4.1 Headline result

Athanor scores **95.7%** on the ARC-AGI-2 public evaluation at an average cost of **\$3.12 per task**, computed across the 119-checkpoint release bundle (one task, `b6f77b65`, excluded for data validity).

Algorithm 1 Inter-Context Artifact Exchange.

Require: puzzle P , threshold τ , max iterations T

```
1:  $C \leftarrow \text{INIT}(P)$  ▷ fresh solver context
2:  $h \leftarrow \emptyset$  ▷ distilled research state
3: for  $t = 1, \dots, T$  do
4:   extend  $C$  with one solver step (reasoning, exploration code, possible candidate)
5:   if  $|C| > \tau$  then
6:      $h \leftarrow \text{COMPRESS}(C)$ 
7:      $C \leftarrow \text{RESUME}(h, P)$ 
8:   end if
9:   if solver emitted candidate artifact  $a = (\text{hyp}, \text{code}, \text{preds}, \text{train\_acc})$  then
10:     $v \leftarrow \text{REVIEW}(a)$  ▷ fresh reviewer context; sees  $a$  only
11:    if  $v = \text{APPROVE}$  then
12:      return  $a$ 
13:    else if  $v = \text{REJECT}$  then
14:       $h \leftarrow \text{COMPRESS}(C, v)$  ▷ atomic: critique enters via compression
15:       $C \leftarrow \text{RESUME}(h, P)$ 
16:    else if  $v = \text{EXPAND\_CANDIDATES}$  then
17:      extend  $C$  with directive to emit a second candidate prediction
18:    end if
19:  end if
20: end for
```

System	Score	Cost/task	Method
Confluence [7]	97.9%	\$11.77	Program synthesis + vote
Squeeze-Evolve [22]	97.5%	\$5.93	Evolutionary recombination
Imbue Darwinian Evolver [13]	95.1%	\$8.71	Evolutionary search
Athanor (this work)	95.7%	\$3.12	Code-verified hypothesis + reviewer

Table 2: ARC-AGI-2 public evaluation: comparison with systems above 95%.

4.2 Hard-case analysis: the gap over pure chain-of-thought

Aggregate accuracy and cost characterize the system on the full benchmark. A complementary question is what an agentic scaffold buys over pure CoT prompting on the puzzles where CoT alone fails. We answer this against the corpus described in Section 2 — 10,529 logged attempts across 58 CoT-only configurations on the public evaluation set, snapshotted on 2026-04-12 [3]. The methodology details for the snapshot, including which configurations are included and the exact attempt counts, are documented in the open-source repository accompanying this paper.

We isolate two subsets of the public evaluation set in which the CoT-only corpus is informative.

Public-corpus frontier ($n = 9$ test-output pairs). We define the public-corpus frontier as the set of test-output pairs ($\text{task_id}, \text{test_index}$) for which no CoT-only configuration in the snapshot has produced a correct output across all 10,529 logged attempts. This frontier contains 9 pairs. Athanor solves 8 of them, an exclusive-solve ratio of 89% on the hardest CoT-only frontier in the corpus. Each of the 8 exclusive solves is a test pair that no single-model CoT configuration in the snapshot has ever produced.

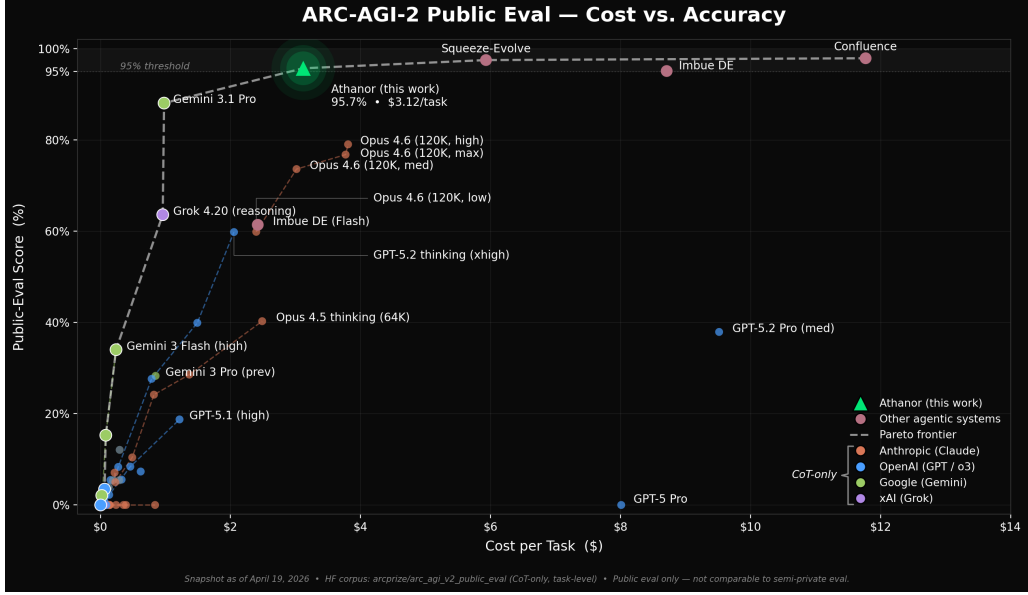


Figure 4: Cost per task vs. accuracy on the ARC-AGI-2 public evaluation. Athanor occupies the Pareto frontier above 95%. The shaded background shows the corpus of CoT-only model runs from huggingface.co/datasets/arcprize/arc_agi_v2_public_eval (10,529 logged attempts); no CoT-only configuration in that corpus exceeds 88% at any cost.

claude-opus-4-6 baseline isolation ($n = 22$ test-output pairs). A second subset isolates the contribution of *this scaffold over the same base model*. The HF corpus contains 8 separate claude-opus-4-6 CoT configurations (varying thinking-effort and prompt budget). We define the Opus-baseline hard set as the set of pairs that none of these 8 configurations has solved across their combined attempts. This set contains 22 pairs. Athanor, which itself runs on claude-opus-4-6 as the solver model, solves **15** of them. Because the same base model is used in both the baseline and this work, the gap is attributable to the scaffold rather than to model capability; equivalently, the scaffold recovers generalization on 68% of pairs that pure CoT on the same model never solves.

Scope. Both numbers are scoped to the CoT-only HF corpus. They do *not* characterize the gap against other agentic ARC-AGI-2 systems (Confluence, Squeeze-Evolve, the Imbue Darwinian Evolver, Poetiq), which do not publish per-pair attempts in a uniform format. The claim being made is narrower: on puzzles where pure CoT — including over a thousand attempts at the strongest single-model CoT configuration of the same base model — has not produced a correct test output, an agentic scaffold of the kind described in Section 3 can.

4.3 Methodology and reproducibility

The headline aggregate of Section 4.1 is computed across the 119-checkpoint release bundle that accompanies this paper. We surface three methodology details that affect how the number should be read, in the spirit of the ARC Prize Foundation’s cross-run reporting expectations and to allow third parties to audit our scoring decisions against the per-task traces.

Per-run scoring compliance. Inside a single run, the solver’s final submission tool may emit up to two candidate output grids per test example. This is the standard ARC-AGI-2 “2-attempts” allowance and is enforced by the orchestrator: the second attempt is gated on the reviewer issuing EXPAND_CANDIDATES. No run in the release bundle exceeds two candidates per test example.

Cross-run selection. The 119 release checkpoints are inference traces collected during the development period (March–April 2026). For each puzzle, the release checkpoint is the best-scoring run from that period, breaking ties by lowest inference cost. This is a development-then-select-best pattern, distinct from single-submission scoring: a single uniform run of the current configuration on all 120 public-eval puzzles would not necessarily reproduce every solved outcome. The reported 95.7% aggregate is the upper envelope across development runs on the public evaluation set, not a single-seed score. We emphasize this distinction because public-evaluation numbers across ARC-AGI-2 systems are not always reported with explicit selection methodology, and direct head-to-head comparison of public-eval aggregates therefore conflates several axes (cost configuration, run count, selection rule) that the per-task release bundle separates out.

Reproducibility. Re-running a small subset of the hardest puzzles in mid-April 2026 with a byte-identical system prompt and tool configuration produced worse outcomes on some of those puzzles. We attribute this to a combination of provider-side model nondeterminism and the genuine variance of search-style agentic loops. Each release checkpoint embeds its exact configuration in `state.config`; loading a checkpoint restores the precise settings for that run, but the model-side nondeterminism is outside our control. Practitioners reproducing the headline aggregate should therefore expect single-seed accuracy below the upper-envelope figure.

Auditable artifact. We release the 119 inference traces as full event-level checkpoints (system prompt, every tool call and tool result, reviewer verdicts, compressed checkpoints) so that the per-task selection underlying the aggregate is third-party auditable. Among the agentic ARC-AGI-2 systems we are aware of, this is, to our knowledge, the most detailed inference-trace artifact publicly released alongside a benchmark result.

One excluded puzzle. Puzzle `b6f77b65` is excluded from the release bundle because of a data-validity concern in the underlying ARC-AGI-2 public release; we therefore report aggregates over 119 rather than 120 tasks. The methodology and the methodology section of the open-source repository document this exclusion in detail.

5 Discussion

5.1 Why does this cost less?

The headline cost gap is approximately $2\times$ against Squeeze-Evolve at comparable accuracy (\$3.12 vs \$5.93 per task). We attribute this gap primarily to the verification mechanism of Section 3.1, with a secondary contribution from the single-thread organization that obviates population-level search. Two arguments support this attribution.

Empirical isolation. Among the four agentic systems in Section 2, Squeeze-Evolve is the only one that performs no code execution at any stage; Confluence and the Imbue Darwinian Evolver both execute code, but in the program-scoring role rather than the verification role. The cost gap that Athanor recovers against Squeeze-Evolve is therefore most cleanly read as the value of moving per-step verification from token-space reasoning into Python execution. Athanor is also \$5–\$6 per task cheaper than the two execution-as-program-scorer systems, which is the secondary contribution of single-hypothesis refinement relative to population search.

Mechanistic argument. The token economy of the verification step itself is structurally favorable: a sub-rule check that takes a CoT system several hundred tokens of careful enumeration is, in Python with a persistent runtime, a one-line predicate plus a Boolean output. Across the dozens of sub-rule checks a hard ARC puzzle requires, this per-check ratio compounds. The

persistent-runtime design choice of Section 3.1 is what allows the per-check predicate to be cheap; without it, re-import and re-parse cost would erode the gap.

We do not claim a mechanistic decomposition of the gap into specific token-budget contributions per mechanism — a proper ablation (Section 5.3) would be required. The argument here is consistent with the headline numbers and with the per-call token figures of Section 3.1, but it is not a controlled measurement.

5.2 What transfers beyond ARC?

Two of the three mechanisms in Section 3 are not specific to ARC.

Artifact-only review (Section 3.2) is a pattern for any agentic loop in which a candidate solution can be expressed as a small set of deliverable artifacts. The pattern requires only that (a) the artifacts are sufficient for an outside reader to reconstruct the underlying task and judge correctness, and (b) a model from a different family is available. It applies to code-generation loops, formal proof attempts, deep-research summaries, and any autoresearch setting in which a generation step can be separated from a verification step. The cross-family choice is the part most likely to generalize: shared distillation lineage within a model family is a structural property of the current foundation-model market, not a specific quirk of the Claude/Gemini pair.

Inter-Context Artifact Exchange (Section 3.3) is a pattern for any agent that needs to make progress across context-window boundaries on a single task. The fixed-schema distilled research state is the part to adapt: fields like *verified invariants*, *failed approaches*, and *open questions* carry over to scientific or engineering tasks; what changes is the domain vocabulary the schema is filled with. The atomic compression-on-rejection property generalizes wherever a verification step can produce structured feedback.

A related question is how much of the abstraction in ARC is visual versus symbolic. Hu et al. [12] take the strong vision-centric position, framing ARC as an image-to-image translation problem and training a Vision Transformer from scratch on ARC data with test-time training; their VARC model reaches 60.4% on ARC-AGI-1, substantially outperforming prior from-scratch methods. We read this result as evidence that visual structure carries genuine signal — and we feed the solver rendered PNG grids alongside the text representation for exactly this reason — but also as evidence that visual structure alone is not sufficient at the current frontier. On ARC-AGI-1, 60.4% is well below the 95%+ scores reached by agentic systems that combine visual perception with explicit hypothesis refinement, and ARC-AGI-2 (the harder sequel benchmark used throughout this paper) widens that gap further. We do not have ablation evidence on the incremental value of the PNG modality in our own system; the position we can defend is narrower: vision and symbolic verification appear complementary, and the cost-efficient frontier on ARC-AGI-2 currently sits with systems that use both.

The first mechanism — code as verification — is more bound to the structure of the underlying task. ARC sub-rules have the convenient property that they can be phrased as predicates over small grids and falsified by short executable checks. Domains in which sub-claims are not so directly executable (legal reasoning, qualitative scientific inference) would need a different verification primitive. The general claim that survives is weaker: where a deterministic verification primitive is available, paying the design cost of separating exploratory from final use of that primitive can recover meaningful inference cost.

5.3 Limitations

We surface five limitations of the present work.

No verified semi-private result. Section 4.3 is restricted to public evaluation. Public-eval numbers are not directly comparable across systems and may reflect tuning on known tasks.

A verified semi-private submission, in the format and infrastructure used by the ARC Prize Foundation, is the appropriate next step before claiming generalization beyond the public set.

No mechanism ablation. The cost-gap attribution of Section 5.1 is mechanistic rather than measured. A proper ablation — separately disabling the artifact-only reviewer, the ICAE self-handover, and the persistent-runtime exploration tool — is required to attribute the gap quantitatively to specific mechanisms.

Cross-run selection. The 95.7% aggregate is the upper envelope across development runs, not a single-seed result (Section 4.3). Other ARC-AGI-2 systems likely apply similar selection during their own development; none have published per-task release bundles that would make the selection visible. The honest comparison is therefore between Athanor’s auditable bundle and other systems’ aggregate-only reports, with the methodology asymmetry flagged rather than concealed.

Hard-case scope. The frontier and Opus-baseline numbers in Section 4.2 are scoped to the CoT-only HF corpus. They do *not* establish that Athanor solves pairs that other agentic systems (Confluence, Squeeze-Evolve, Imbue Darwinian Evolver, Poetiq) cannot solve. Such a claim would require uniform per-pair attempt logs from those systems, which are not currently published.

Provider-side nondeterminism. Re-runs of the hardest subset with byte-identical configuration produced different outcomes (Section 4.3). This affects reproducibility of any single checkpoint and limits the precision of small-sample claims. Practitioners reproducing the headline aggregate should plan for variance.

6 Conclusion

We have presented Athanor, an agentic system for ARC-AGI-2 that organizes its loop around three interlocking mechanisms: code execution as a verification side-channel rather than as a program-scoring engine; an independent artifact-only reviewer running on a model from a different family than the solver; and Inter-Context Artifact Exchange, a fixed-schema continuation mechanism that lets a single solver thread iterate across context-window boundaries. The combination scores 95.7% on the ARC-AGI-2 public evaluation at \$3.12 per task — the lowest cost-per-task in the $\geq 95\%$ band, and approximately $2\times$ cheaper than the strongest comparable-accuracy system that achieves its score without code execution. On the hardest subsets of the public evaluation set — pairs that no CoT-only configuration in a 10,529-attempt public corpus has solved, and pairs that no Opus-4.6 CoT baseline configuration has solved — the same scaffold over the same base model recovers the majority of cases.

The contribution we hope is most transferable is the artifact-vs-reasoning-chain distinction itself: maintaining isolated agent contexts and exchanging only deliverable artifacts is a generic pattern for keeping verification and search honest, and the ICAE schema is a concrete way to keep that pattern coherent across context resets. We release the 119 inference traces as an auditable record of the per-task scoring underlying these claims.

Acknowledgments

References

- [1] Anthropic. The claude 3 model family: Opus, sonnet, haiku. 2024. URL https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf. Model card.
- [2] Anthropic. Claude Platform release notes, 2026. URL <https://platform.claude.com/docs/en/release-notes/api>. Claude API release-notes log; Opus 4.6 launched 5 February 2026 with 200,000-token default window and 1M beta, with 1M moved to general availability at standard pricing on 13 March 2026.
- [3] ARC Prize Foundation. Arc-agi-2 public evaluation attempts (huggingface dataset), 2026. URL https://huggingface.co/datasets/arcprize/arc_agi_v2_public_eval. Aggregated CoT-only model runs on the public evaluation set.
- [4] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- [5] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- [6] François Chollet et al. Arc-agi-2: A new challenge for frontier AI reasoning systems, 2024. URL <https://arcprize.org/arc-agi-2>. ARC Prize Foundation.
- [7] Confluence Labs. Confluence: Program synthesis with voting for arc-agi-2, 2026. URL <https://github.com/confluence-labs/arc-agi-2>. GitHub repository.
- [8] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.
- [9] Gemini Team, Google. Gemini: A family of highly capable multimodal models, 2023.
- [10] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [12] Keya Hu, Ali Cy, Linlu Qiu, Xiaoman Delores Ding, Runqian Wang, Yeyin Eva Zhu, Jacob Andreas, and Kaiming He. ARC is a vision problem! *arXiv preprint arXiv:2511.14761*, 2025.
- [13] Imbue. The darwinian evolver: Population-based program search for arc-agi-2, 2025. URL https://github.com/imbue-ai/darwinian_evolver. GitHub repository.
- [14] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks — a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.
- [15] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:157–173, 2024.

- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegr-eff, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] Poetiq AI. Announcing poetiq’s submission to arc-agi, 2025. URL https://poetiq.ai/posts/arcagi_announcement/. Blog announcement.
- [18] Poetiq AI. Poetiq arc-agi solver (source release), 2025. URL <https://github.com/poetiq-ai/poetiq-arc-agi-solver>. GitHub repository.
- [19] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling trans-former inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
- [20] SGLang and Miles Team. DeepSeek-V4 on Day 0: From fast inference to ver-ified RL with SGLang and Miles, 2026. URL <https://www.lmsys.org/blog/2026-04-25-deepseek-v4/>. LMSYS blog post, April 25 2026.
- [21] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [22] Squeeze-Evolve Authors. Squeeze-evolve: Evolutionary recombination for abstract reason-ing, 2026. URL <https://arxiv.org/abs/2604.07725>.
- [23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [24] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large lan-guage models. *arXiv preprint arXiv:2305.16291*, 2023.
- [25] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [26] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Full exploration code from release run 6ffbe589

The exploratory verification snippet referenced in Section 3.1 and abbreviated in Listing 1, reproduced verbatim from `release_runs/6ffbe589.json`, iteration 0.

```
# Check all rotations for color 6
pattern = np.array(train_samples[0]['input'])[0:13, 6:19]
mask_in = (pattern == 6)

for k in range(4):
    print(f"k={k} ({k*90}deg CCW) matches?",
          np.array_equal(np.rot90(mask_in, k), mask_in))
```

Tool result (returned to the solver in ~39 output tokens):

```
Runtime: reused (id=R1, step=11)
STDOUT:
k=0 (0deg CCW) matches? True
k=1 (90deg CCW) matches? False
k=2 (180deg CCW) matches? True
k=3 (270deg CCW) matches? False
```

B Full reviewer REJECT response from release run faa9f03d

The full reviewer response sketched in Section 3.2 and Listing 2, reproduced verbatim from `release_runs/faa9f03d.json`, iteration 3 (orchestrator-added preamble and postscript trimmed).

The candidate at this point passed all four training pairs.

An analysis of the training examples reveals a beautifully consistent set of rules for the path routing and intersections, but your current implementation contains two critical over-constrained heuristics that cause it to completely fail on Test 0.

Your Independent Hypothesis & Deliberation

- 1. The Flip Marker Logic is Sound, but the Classification is Buggy:*
In Train 1 and Train 3, a yellow marker (4) acts as a "flip" instruction. It flips the corner segment that lies on the same axis as the marker to the opposite direction (e.g., if the marker is DOWN from the corner, it erases the UP segment and draws a DOWN segment to the boundary).

In Test 0, there are two yellow markers: at (3,0) and (4,10).

- (3,0) is LEFT of the corner at (3,1). It should erase the RIGHT segment (3,1)–(3,4) and draw a LEFT segment to (3,0).*
- (4,10) is DOWN of the corner at (3,10). It should erase the UP segment (3,10)–(0,10) and draw a DOWN segment to (11,10).*

The Bug: Your code loops over markers_4 and checks if it has an adjacent line cell using `inp[nr,nc]` not in `[0, 2, 4]`. Because the adjacent corners in Test 0 are marker 2s, your code thinks there are no adjacent line cells. It misclassifies BOTH yellow markers as `extend_markers`. Marker 2 IS a valid part of the line.

- 2. The Gap Fill Logic is Unconstrained and Undoes Flips:*
In Test 0, the only real gap of 0s between 2s is between (3,4) and (3,7) (at cols 5 and 6). The other 2s on row 3 ((3,1) to (3,4) and (3,7) to (3,10)) are ALREADY connected by Orange 7s.

The Bug: Your Step 4 gap fill blindly iterates over all pairs of 2s on the same row/col and unconditionally fills the space between them. Because of this, it filled the entirety of row 3 from col 1 to 10 with Orange. Worse, if your flip logic had correctly erased (3,2) and (3,3), your gap fill would have seen the newly created 0s and immediately filled them right back in, undoing the flip. Gap fill should only apply to spaces that were empty in the original input grid.

- 3. The Intersection Swapping Logic is Perfect:*
When the Orange snake is correctly routed (going down col 1, col 7, and col 10), it will intersect the Pink line three times. Your intersection swapping logic elegantly handles this, creating a perfect alternating over/under weave.

Verdict
VERDICT: REJECT

Repair Plan

To fix the candidate generation, prioritize these specific corrective actions:

1. *Fix Flip Marker Classification:* Update your `has_line` check so that a cell is recognized as part of a line if it is not 0 and not 4. Marker 2 must be treated as a valid adjacent line cell.
2. *Fix Corner Finding in Flip Logic:* When tracing from the flip marker to find the corner, look for an adjacent cell that is either the resolved line color `lc` OR 2. (Right now, you only check `inp[nr, nc] == lc`.)
3. *Constrain Gap Fill:* Modify your Step 4 gap fill so that it *ONLY* connects two 2s if the space between them in the `inp` grid was strictly 0. Do not connect 2s if there were already line cells (like 7) between them.
4. *Validate Test 0 Output:*
 - The segment (3,2) to (3,3) must be erased (0).
 - The segment (0,10) to (2,10) must be erased (0).
 - The gap at (3,5) and (3,6) must be filled (7).
 - A new segment must go down from (5,10) to (11,10) (7).
 - Row 7 should show the correct alternating weave: Orange crosses Pink at col 1 and 10, while Pink crosses Orange at col 7.

C Full distilled research state from release run faa9f03d

The full distilled research state referenced in Section 3.3, reproduced verbatim from `release_runs/faa9f03d.js` immediately after iteration 0 of solving puzzle `faa9f03d`. This is the artifact that initialized the solver's iteration 1 context window in lieu of the raw transcript.

```
# Memory Checkpoint for ARC Puzzle

## 1. Puzzle Understanding & Verified Invariants

### Core Structure
Each input grid (12x12) contains L-shaped/zigzag paths made of colored lines.
The paths have:
- Marker color 2: Appears at corners of L-shaped line segments. These are
  "corner markers" placed where a horizontal arm meets a vertical arm of the
  same path.
- Marker color 4: Appears adjacent to corners, indicating that one arm
  should be flipped (reversed in direction).
- Both 2 and 4 can appear in the same puzzle (test example has both).

### Transformation Rule (High-Level)
1. Replace markers (2/4) with the correct line color
2. For marker 4: flip the arm direction (remove cells on wrong side, extend
  on correct side)
3. Fill gaps in lines to make them continuous
4. Resolve intersections where two different-colored paths cross

### Training Examples Summary
| Ex | Marker | Line Colors | Intersections |
|----|-----|-----|-----|
| 0 | 2 only | 1, 3 | 3->1 at (3,3), (5,4), (8,6) |
| 1 | 4 only | 3, 7 | 7->3 at (2,10); gap 3 at (9,2) |
| 2 | 2 only | 1, 3, 6, 7 | 7->3 at (3,2); 1->6 at (4,9) |
| 3 | 4 only | 1, 6 | Keep 1 at (2,1); gap->1 at (2,6) |

### Test Example
- Colors: 2, 4, 6, 7 (both marker types present)
- Marker 2 at: (3,1), (3,4), (3,7), (3,10), (9,1), (9,7) - all on rows 3 and 9
- Marker 4 at: (3,0) - left of corner (3,1); (4,10) - below corner (3,10)
- Color 6: horizontal line on row 7 (gap at col 7)
- Color 7: forms bracket structure with vertical arms and horizontal connectors
```

```

### What Changes Between Input and Output
1. Markers (2, 4) -> replaced with line colors
2. Gaps (0 cells) between same-colored cells -> filled
3. Wrong-color cells at intersections -> swapped to other path's color
4. For marker 4: arm cells on wrong side -> removed (set to 0); arm extends
   in indicated direction
5. Some cells may be removed (wrong-side arm cells)

## 2. Experiments & Learnings

### Marker Identification
- Marker color is always 2 or 4
- Each marker's line color = most common adjacent non-zero, non-marker color
- Ex0: all markers -> color 3; Ex1: marker -> color 3; Ex2: some -> 3, some
  -> 1; Ex3: marker -> color 6

### Marker 4 Arm Flipping
The marker 4 cell is adjacent to a corner cell. The direction from
corner->marker = the NEW arm direction.
- Ex1: marker 4 at (2,9), corner at (2,8)=3. Flip direction: RIGHT. Remove
  3s going LEFT from corner. Extend RIGHT to boundary.
- Ex3: marker 4 at (3,10), corner at (2,10)=6. Flip direction: DOWN. Remove
  6s going UP. Extend DOWN to boundary.
- When removing wrong-side arm: traverse ALL cells in opposite direction
  (even through 0-gaps), removing same-color cells.
- When extending correct-side arm: extend to grid boundary, overwriting ALL
  cells (including other colors).

### Between-Markers Fill
For markers of the same color on the same row/column, fill ALL cells between
them with that color. This creates the marked path's arm segments.
- CRITICAL: Do NOT extend beyond the last marker. Only extend from "orphan"
  markers (those without another marker in the same direction on the same
  row/col) through existing same-color cells.

### Intersection Resolution Rules (Complex - Main Challenge)
For intersections of MARKED arm with UNMARKED arm:
- Rule: output = NOT-original color (swap)
For intersections of TWO UNMARKED arms:
- NO swap - keep original color
For 0-gap cells where both H-fill and V-fill conflict:
- Need connectivity analysis: fill with the color that prevents path
  disconnection.

### Gap Filling Heuristic
Simple 1-cell gap fill (fill 0 between two same-color cells) is too
aggressive. Need additional check:
- For VERTICAL gap: don't fill if BOTH the above and below cells have
  same-color HORIZONTAL neighbors.
- For HORIZONTAL gap: don't fill if BOTH left and right cells have
  same-color VERTICAL neighbors.

## 3. Hypothesis Evolution

### Failed Approaches
1. "Unmarked path always wins" - fails for Ex1 where marked (3-path) wins
   at (2,10)
2. "Marked path always wins" - fails for Ex0 where unmarked (1-path) wins
3. "Inner path wins" - correct conceptually but very hard to determine
   "inner" programmatically
4. "Shorter arm wins" - partially works but has tie-breaking issues
5. "Horizontal always wins" / "Vertical always wins" - inconsistent across
   examples
6. "Fill everything, then swap non-original" - works for marked/unmarked
   intersections but not for unmarked/unmarked or 0-gap cases

### Current Best Understanding
The NOT-original swap rule works when at least one arm at the intersection
is a "marked arm" (between markers or created by flipping). When neither
arm is marked, keep original. For 0-gap conflicts, use connectivity analysis.

### What's Wrong with Current Implementation

```

```

1. Syntax error: Used Unicode minus sign instead of ASCII hyphen in code
2. Endpoint extension logic: Extended too far beyond markers
3. Intersection detection: Need to check original grid values to find
   perpendicular arms at marked cells
4. 0-gap conflict resolution: Need connectivity-based tiebreaker

### Confidence: 3/5
The conceptual framework is solid but implementation has edge cases.

## 4. Verified Invariants (ALL training examples)
- Grid size always 12x12
- Marker colors are always 2 and/or 4
- Marker 2 = corner marker; marker 4 = direction indicator
- Output never contains colors 2 or 4
- Each line segment is 1 cell wide
- Gaps in lines are always 1 cell wide
- Each non-zero, non-marker cell in the output belongs to exactly one
  horizontal or vertical line segment
- At each intersection of two paths, one path is continuous (winner) and
  the other is interrupted (gap)

## 5. Current State & Next Steps

### Where I Am
I have the correct conceptual framework but the last implementation had a
syntax error (Unicode minus). I need to fix and resubmit.

### What to Try Next
1. Fix syntax error (Unicode minus -> ASCII hyphen)
2. Fix endpoint extension (don't extend past other markers)
3. Implement proper intersection resolution with NOT-original rule for
   marked arms
4. Implement connectivity-based tiebreaker for 0-gap conflicts
5. Test on all training examples
6. Submit

```